Assignment 4: Relational Database vs. NoSQL Database

Project context: Hotel Management System

1. Introduction

Relational databases and NoSQL databases are two common ways to store application data. The correct choice depends on the structure of the data, the need for transactions, expected traffic, and how the system must grow.

2. Comparison

Feature	Relational database (RDBMS)	NoSQL database
Data structure	Tables with rows, columns, and defined relationships.	Document, key-value, wide-column, or graph structures.
Schema	Usually defined before inserting data; changes are managed through schema updates.	Often flexible; records can have different fields depending on the model.
Scalability	Commonly scales vertically; many systems also support read replicas and distributed scaling.	Often designed for horizontal scaling across multiple servers.
Consistency	Strong transactional consistency is common; ACID transactions support reliable updates.	Varies by product; may offer strong or eventual consistency, and many support transactions.
Performance	Good for joins, structured queries, and transaction-heavy workloads.	Can be very fast for access patterns suited to its data model and partitioning.
Relationships	Foreign keys and joins support related data.	Relationships may be embedded, duplicated, or handled by application logic, depending on the database.
Best use cases	Banking, reservations, inventory, payroll, order processing.	Large-scale event data, flexible content, caching, product catalogs, graph relationships.

3. Why organizations choose different technologies

Organizations choose a database according to their business needs rather than because one technology is always better. A system that needs accurate payments and bookings usually benefits from transactions and constraints. A system that stores millions of changing event records may prefer a distributed NoSQL database. Cost, staff skills, reporting needs, existing software, availability, and data security also affect the decision.

4. Real-world examples

Example	Suitable database approach	Reason
Hotel reservation system	Relational database such as MySQL or PostgreSQL.	Guest, room, booking, and payment records have clear relationships and require reliable transactions.
Online shopping platform	Often a combination of relational and NoSQL databases.	Orders and payments need transactional consistency; product search, sessions, or activity data may use other data stores.
Social media activity feed	A NoSQL or distributed data store may be used for some components.	Large volumes of posts, events, and feeds may need flexible schemas and horizontal scaling.
Banking application	Relational database is common for core account transactions.	Balances and transfers require strong consistency, auditability, and transaction controls.
Content management system	Relational or document database, depending on the application.	Structured content may fit tables, while variable document fields can fit document storage.

5. Applying the comparison to this project

My Hotel Management System uses MySQL because guests, rooms, bookings, payments, and services have clear relationships. Foreign keys protect those relationships, CHECK constraints validate values, and SQL joins make reports easier. A NoSQL database could be useful for a separate feature such as large-scale website click logs, but it is not necessary for this small transactional project.

6. Conclusion

Relational and NoSQL databases solve different problems. Relational databases are a strong choice for structured, related, transaction-based data. NoSQL databases can be suitable for flexible data models and distributed workloads. Organizations select technology by matching its strengths to their requirements.